



Third Year Engineering

21BTCS503 - Introduction to Data Science

Class - T.Y. Core (SEM-V)

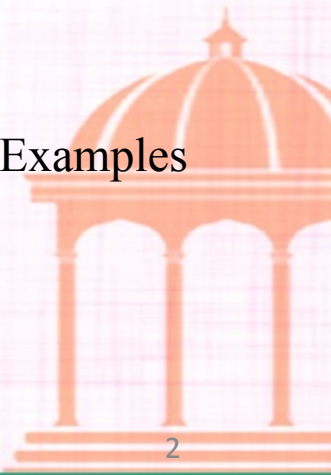
Unit – V(Advance Data Analysis)





Syllabus: Advance Data Analysis

- Decision Trees: What Is a Decision Tree?
- Entropy, The Entropy of a Partition, Creating a Decision Tree,
- Random Forests Neural Networks : Perceptrons,
- Feed-Forward Neural Networks, Backpropagation,
- Example: Defeating a CAPTCHA MapReduce : Why MapReduce? Examples like word count and matrix multiplication





Decision Trees: What is a Decision Tree?

A decision tree is a supervised learning algorithm that is used for both classification and regression tasks. It is a tree-like model of decisions and their possible consequences, including chance event outcomes, resource costs, and utility. Here's a detailed explanation of what a decision tree is:

What Is a Decision Tree?

Definition: A decision tree is a flowchart-like structure where an internal node represents a feature (or attribute), a branch represents a decision rule, and each leaf node represents the outcome (class label or value).

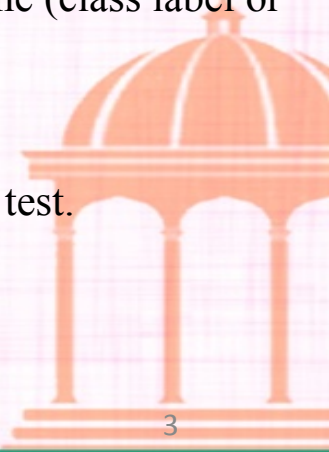
Components:

Root Node: Represents the entire dataset and is split into subsets based on an attribute test.

Internal Nodes: Represent features or attributes on which the data is split.

Branches: Represent the outcome of a decision rule applied to a feature.

Leaf Nodes: Represent the final outcome or class label.





Splitting: The process of dividing a node into two or more sub-nodes based on a feature that results in the best split according to a certain criterion (e.g., information gain, Gini impurity).

Decision Criteria:

Information Gain: Measures how much information a feature gives us about the class. It's based on the concept of entropy from information theory.

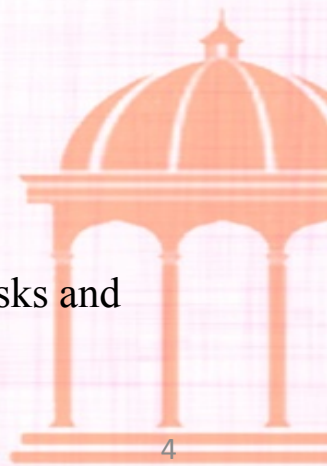
Gini Impurity: Measures the frequency at which any element of the dataset would be misclassified when it is randomly labeled.

Algorithm:

ID3 (Iterative Dichotomiser 3): Uses information gain to decide the splitting of nodes.

C4.5: An extension of ID3 that handles both categorical and continuous features.

CART (Classification and Regression Trees): Uses Gini impurity for classification tasks and mean squared error for regression tasks.





Pruning: A technique used to avoid overfitting by removing nodes that provide little power to classify instances. There are two types of pruning:

Pre-Pruning (Early Stopping): Stop growing the tree earlier based on certain conditions.

Post-Pruning: Remove branches from a fully grown tree that do not provide additional power.

Advantages:

Easy to understand and interpret.

Can handle both numerical and categorical data.

Requires little data preprocessing.

Disadvantages:

Prone to overfitting if not pruned.

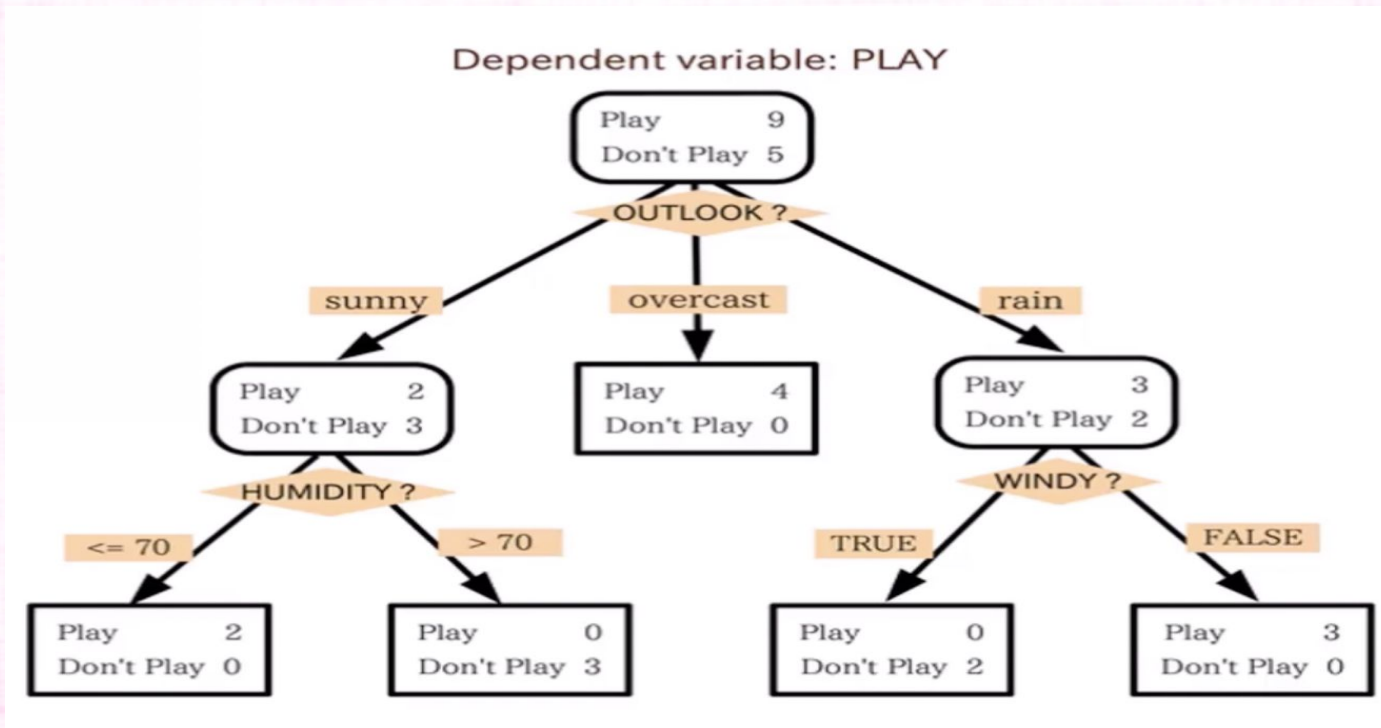
Can be biased towards features with more levels.

Small changes in data can lead to a completely different tree structure.





An example:





Consider a simple example of a decision tree used to decide whether to play tennis based on weather conditions:

Root Node: Weather condition (Sunny, Overcast, Rain).

Internal Nodes:

If Sunny: Check humidity (High, Normal).

If Overcast: Directly play tennis.

If Rain: Check wind (Strong, Weak).

Leaf Nodes:

Sunny + High Humidity: Don't play tennis.

Sunny + Normal Humidity: Play tennis.

Overcast: Play tennis.

Rain + Strong Wind: Don't play tennis.

Rain + Weak Wind: Play tennis.





Splitting: The process of dividing a node into two or more sub-nodes based on a feature that results in the best split according to a certain criterion (e.g., information gain, Gini impurity).

Splitting Criteria

Purpose:

Determines the feature and threshold to split data at each node for maximum separation.

Common Criteria:

Gini Impurity:

Measures impurity of a node. Lower Gini values indicate better splits.

$\text{GiniIndex} = 1 - \sum p_j^2$, where p_j is the probability of class j .

Entropy (Information Gain):

Measures information gain after a split. Higher information gain indicates a better split.

Variance Reduction (for regression):

Splits that reduce variance in the target variable are preferred.





MIT School of Computing

Department of Computer Science and Engineering



MIT-ADT
UNIVERSITY
PUNE, INDIA

Stopping Conditions:

Maximum tree depth.

Minimum number of samples per node.

No further improvement in splitting criteria.

Advantages

- Easy to understand and interpret.
- Handles both numerical and categorical data.
- Requires minimal data preprocessing.

Disadvantages

- Prone to overfitting on training data.
- Sensitive to small changes in data (high variance).

Applications

Customer segmentation, disease diagnosis, fraud detection, etc.





Challenges with Missing or Inaccurate Data

- **Loss of Information:** Missing or incorrect data reduces the model's ability to make accurate predictions.
- **Bias:** Missing or inaccurate values can introduce bias, skewing the model's understanding of the data.
- **Decreased Performance:** Poor handling of missing or inaccurate data can lead to underfitting or overfitting.
- **Distorted Splits:** In Decision Trees, inaccurate data affects how the tree splits the data, leading to incorrect decisions.
- **Preprocessing Complexity:** Requires additional steps like imputation or data cleaning, increasing time and effort.
- **Reduced Generalization:** Models trained with incomplete or erroneous data often perform poorly on unseen data.



Solutions

- **Imputation:** Replace missing values with mean, median, or prediction-based methods.
- **Data Cleaning:** Correct or remove inaccurate data points.



MIT School of Computing

Department of Computer Science and Engineering



MIT-ADT
UNIVERSITY
PUNE, INDIA

- **Algorithm-Specific Handling:** Use models (e.g., Random Forests, XGBoost) that handle missing values.
- **Drop Data:** Remove instances or features with excessive missing or inaccurate values.





Overfitting in Decision Trees :

Overfitting in **Decision Trees** occurs when the tree becomes too complex, capturing not only the patterns in the training data but also the noise, leading to poor generalization on unseen data.

Signs of Overfitting in Decision Trees:

Deep Tree: A very deep tree with many branches is likely to fit noise in the data.

High Training Accuracy: The tree performs extremely well on training data but poorly on test data.

Too Many Branches: The tree has too many branches, indicating that it is modeling every small detail in the training data.

Causes of Overfitting in Decision Trees:

Excessive Depth: If the tree grows too deep, it creates complex rules that overfit the training data.

Small Data Subsets: When nodes split on too few samples, the model may fit noise rather than meaningful patterns.

Too Many Features: If too many features are considered for splits, the model may overfit to certain features.



Solutions to Prevent Overfitting :

To prevent overfitting in Decision Trees, several techniques can be applied. **Pre-pruning** stops tree growth early by setting limits on depth, minimum samples per leaf, or minimum impurity decrease, while **post-pruning** involves fully growing the tree and then removing branches that don't improve performance. Limiting the **maximum depth** restricts the number of splits, preventing complexity, and setting **minimum samples per split** ensures splits occur only with sufficient data. **Minimum samples per leaf** guarantees each leaf node has enough data to avoid overly specific patterns. Limiting **maximum features** for splits reduces overfitting to certain features, and using **cross-validation** helps select the best tree parameters for better generalization to unseen data.





Bias –Variance Tradeoff

Bias: Error due to overly simplistic models (underfitting). High bias leads to poor performance on both training and test data

Variance: Error due to overly complex models (overfitting). High variance leads to great performance on training data but poor performance on test data.

Tradeoff:

Low bias, high variance = Overfitting (complex models).

High bias, low variance = Underfitting (simple models).





Bias –Variance Tradeoff

Effect of Decision Tree Depth on Bias-Variance Tradeoff:

Shallow Trees (Low Depth):

Bias: High bias (underfitting). A shallow tree makes oversimplified assumptions and may not capture all the important patterns in the data.

Variance: Low variance. Since the tree is simple, it doesn't react strongly to variations in the data.

Result: The model is too simple, leading to poor performance on both training and test data.

Deep Trees (High Depth):

Bias: Low bias (overfitting). A deep tree can capture complex patterns and details in the data, making fewer assumptions.

Variance: High variance. A deep tree is sensitive to small fluctuations in the training data, leading to overfitting.

Result: The model performs very well on the training data but may perform poorly on unseen test data due to its overfitting.

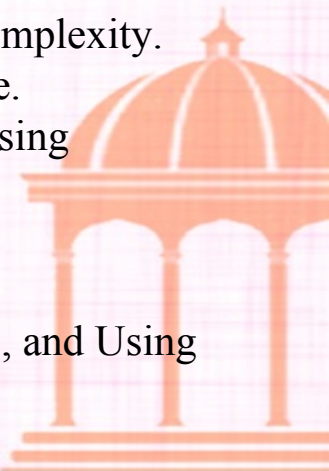


Bias –Variance Tradeoff

Techniques to Balance Bias and Variance in Decision Trees:

- **Cross-validation:** A technique that splits the dataset into subsets, training the model on some and validating it on others. This helps reduce variance and avoid overfitting.
- **Bagging:** A technique that decreases variance in predictions without increasing bias. Bagging makes the difference between training accuracy and test accuracy smaller.
- **Regularization:** Techniques like Lasso or Ridge can reduce variance by penalizing complexity.
- **Ensemble models:** Techniques like Random Forests can help reduce bias and variance.
- **Train region-specific models:** Train an XGBoost regression model for each region, using region-specific data.
- **Prune decision trees:** Fit model parameters, such as pruning decision trees.

Other techniques include: Reducing dimensions, Reducing error, Tuning hyperparameters, and Using proper model performance metrics.





Technique	Controls	Effect on Bias-Variance Tradeoff
Pruning	Depth, size	Reduces variance, slightly increases bias
Minimum Samples per Split	Split size	Simplifies tree, reduces variance
Maximum Features per Split	Features at split	Prevents overfitting, reduces variance
Bagging (e.g., Random Forest)	Model ensemble	Reduces variance, slight increase in bias
Boosting	Sequential learning	Reduces bias, slight increase in variance
Cross-Validation	Model evaluation	Balances bias and variance by tuning hyperparameters



Entropy, The Entropy of a Partition

Entropy is a measure of the disorder or impurity of a dataset in machine learning. It helps decision tree algorithms determine the best attribute to split the data, leading to more accurate classification or prediction.





Entropy, The Entropy of a Partition

- In a binary classification problem, where we have two classes (e.g., "Yes" or "No"), entropy is calculated using the formula:

$$\text{Entropy}(S) = -p(\text{Yes}) * \log_2(p(\text{Yes})) - p(\text{No}) * \log_2(p(\text{No}))$$

Here, $p(\text{Yes})$ and $p(\text{No})$ represent the probabilities of the positive and negative classes in the dataset S . The entropy value ranges from 0 to 1, where 0 indicates a pure class (all samples belong to the same class), and 1 indicates a perfectly mixed class (equal samples from both classes).



Information Gain and Root Node Selection

1. Information Gain (IG)

Definition: Information Gain quantifies how much entropy is reduced after splitting the data on a feature. It helps determine the most relevant feature for splitting the data at a node.

Formula:

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_{(v)})$$

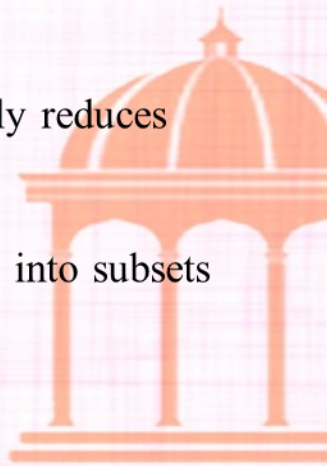
2. Root Node Selection

The feature with the **highest Information Gain** becomes the root node because it maximally reduces uncertainty.

For example:

If "Transaction Amount" has the highest IG, it becomes the root node, splitting transactions into subsets based on amount thresholds.

Subsequent splits are based on the remaining features and their IG.





Challenges of Using Entropy :

1. Class Imbalance

Problem: In fraud detection, fraudulent transactions (positive class) are much rarer than legitimate ones (negative class). This imbalance skews entropy calculations.

Effect: Splits might favor the majority class, leading to a model that poorly detects fraud.

Solution:

Use **class-weighted entropy** to give more importance to the minority class.

Resample the data (oversample fraud cases or undersample legitimate cases).

2. Noisy Data

Problem: Features like transaction time or location may have inconsistencies (e.g., legitimate users may travel or shop online).

Effect: Noisy data can lead to suboptimal splits or overfitting.

Solution:

Preprocess data to filter noise and normalize values.

Use ensemble methods like Random Forests to reduce overfitting from noisy data.





In Fraud Detection –

3.Imbalanced Dataset:

Problem: Fraudulent transactions are much rarer, leading to biased predictions toward the majority class (non-fraudulent).

Solution:

Adjust **class weights** to prioritize fraud.

Apply **anomaly detection** methods.

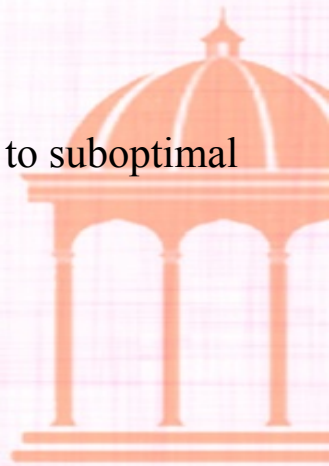
4.Handling Continuous Features:

Problem: Poor threshold selection for continuous features like transaction amount can lead to suboptimal splits.

Solution:

Normalize or **standardize** continuous features.

Consider alternative splitting criteria (e.g., **Gini impurity**).





MIT School of Computing

Department of Computer Science and Engineering



MIT-ADT
UNIVERSITY
PUNE, INDIA

5.Data Sparsity:

Problem: Rare fraud behaviors lead to complex or ineffective trees.

Solution:

Apply **feature engineering**.

Use **ensemble methods** (e.g., Random Forests).





Random Forests

A **Random Forest** is an ensemble machine learning algorithm that builds multiple decision trees and merges them together to improve accuracy and reduce overfitting. It is used for both classification and regression tasks.

How it works:

Random Forests create many decision trees by randomly sampling the data (with replacement) and using a random subset of features for each split.

Each tree in the forest makes an independent prediction, and the final result is determined by aggregating the predictions of all trees (via majority voting for classification or averaging for regression).





Random Forests

Neural Networks (Perceptrons)

Definition:

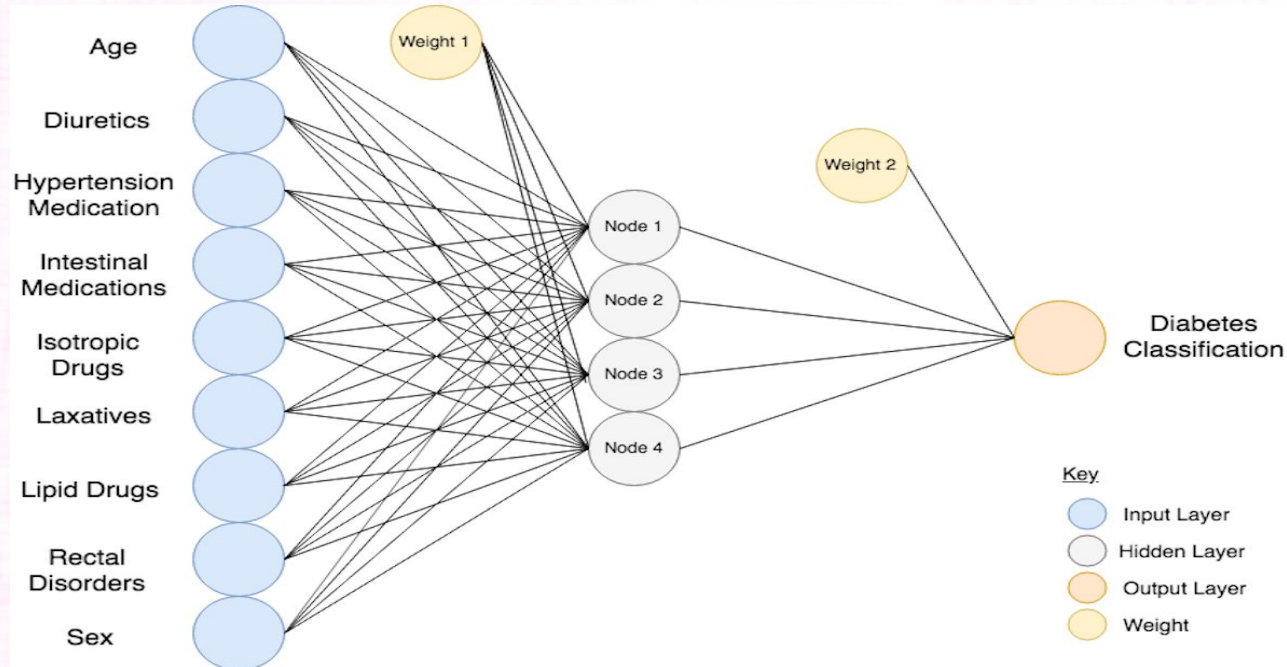
A **Perceptron** is the simplest type of neural network, consisting of a single layer of artificial neurons. It is a binary classifier that maps input features to an output value, typically used for linearly separable problems.





Random Forests Neural Networks

- Combination of Random Forests and Neural Networks.... An example:





Feed Forward Neural Networks

Artificial [Neural Networks](#) (ANNs) have revolutionized the field of machine learning, offering powerful tools for pattern recognition, classification, and predictive modeling. Among the various types of neural networks, the Feedforward Neural Network (FNN) is one of the most fundamental and widely used. In this article, we will explore the structure, functioning, and applications of Feedforward Neural Networks, providing a comprehensive understanding of this essential machine learning model.

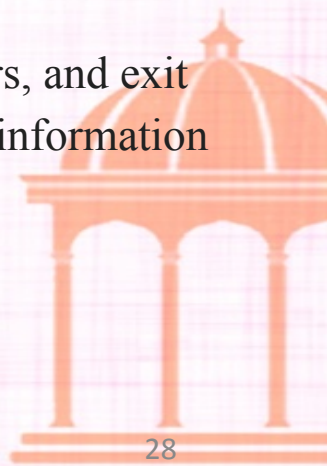
What is a Feedforward Neural Network?

A Feedforward Neural Network (FNN) is a type of artificial neural network where connections between the nodes do not form cycles. This characteristic differentiates it from recurrent neural networks (RNNs). The network consists of an input layer, one or more hidden layers, and an output layer. Information flows in one direction—from input to output—hence the name "feedforward."



Feed Forward Neural Networks

- Feed forward neural networks are **artificial neural networks** in which nodes do not form loops. This type of neural network is also known as a multi-layer neural network as all information is only passed forward.
- During data flow, input nodes receive data, which travel through hidden layers, and exit output nodes. No links exist in the network that could get used to by sending information back from the output node.





Feed Forward Neural Networks

A feed forward neural network approximates functions in the following way:

- An algorithm calculates classifiers by using the formula $y = f^*(x)$.
- Input x is therefore assigned to category y .
- According to the feed forward model, $y = f(x; \theta)$. This value determines the closest approximation of the function.





Feed Forward Neural Networks

Structure of a Feedforward Neural Network :

Input Layer: The input layer consists of neurons that receive the input data. Each neuron in the input layer represents a feature of the input data.

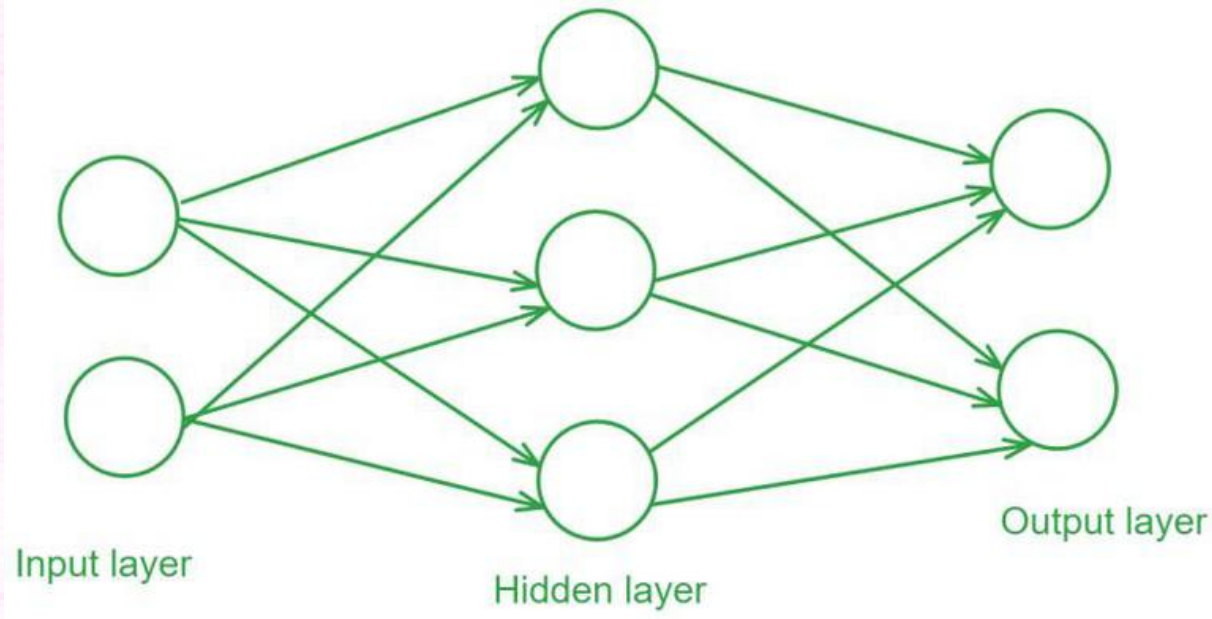
Hidden Layers: One or more hidden layers are placed between the input and output layers. These layers are responsible for learning the complex patterns in the data. Each neuron in a hidden layer applies a weighted sum of inputs followed by a non-linear activation function.

Output Layer: The output layer provides the final output of the network. The number of neurons in this layer corresponds to the number of classes in a classification problem or the number of outputs in a regression problem.

Each connection between neurons in these layers has an associated weight that is adjusted during the training process to minimize the error in predictions.



Each connection between neurons in these layers has an associated weight that is adjusted during the training process to minimize the error in predictions.



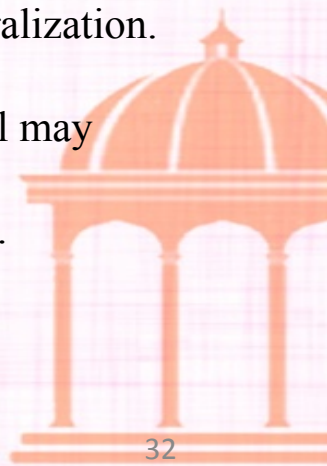


Overfitting in Feedforward Neural Networks :

Overfitting in Feedforward Neural Networks (FNNs) occurs when the model learns to perform well on the training data but poorly on unseen data, typically due to excessive complexity or noise in the data.

Causes:

- **Excessive Model Complexity:** Too many layers or neurons, leading to memorization of data.
- **Insufficient Training Data:** Limited or unrepresentative data, leading to poor generalization.
- **Noise in Data:** Random fluctuations or outliers that the model memorizes.
- **Lack of Regularization:** Without techniques like L2 or L1 regularization, the model may overfit.
- **Too Many Epochs:** Training for too long can cause the model to memorize the data.
- **High Learning Rate:** Large updates can cause overfitting to noise.





Prevention:

- **Regularization:** L2/L1 regularization to penalize large weights.
- **Dropout:** Randomly drop neurons during training to prevent over-reliance on specific features.
- **Early Stopping:** Stop training when validation error increases.
- **Cross-Validation:** Use to ensure the model generalizes well.
- **Simplify the Model:** Reduce the number of layers or neurons.
- **Increase Data:** More or augmented data helps the model generalize.
- **Noise Reduction:** Clean the data by removing outliers.





Generalization:

Generalization refers to a model's ability to perform well on new, unseen data after being trained on a given dataset. A model is said to generalize well if it accurately captures the underlying patterns of the data and can make accurate predictions on data it hasn't encountered before.

Effect of Overfitting on Generalization:

When a model **overfits** the training data, it performs very well on the training set but fails to generalize to new, unseen data. This happens because:

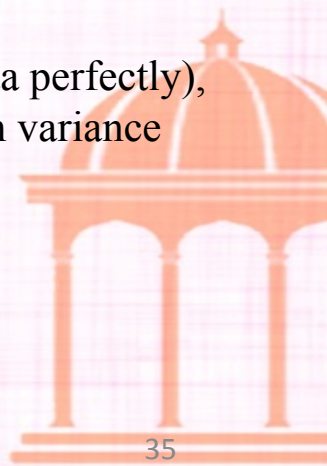
1.Memorization: The model learns the specific details and noise in the training data rather than the underlying patterns. This leads to a **high variance** in the model's predictions, causing it to perform poorly on any data outside the training set.



2.Bias-Variance Tradeoff: In overfitting, the model's bias is low (it fits the training data perfectly), but its variance is high (it reacts too strongly to the idiosyncrasies of the data). This high variance leads to poor generalization.

3.Loss of Predictive Power: Overfitting reduces the model's ability to make predictions on data that isn't exactly like the training data. The model becomes **too complex** and tailors its decision-making to the peculiarities of the training set, resulting in poor **out-of-sample performance**.

4.Bias-Variance Tradeoff: In overfitting, the model's bias is low (it fits the training data perfectly), but its variance is high (it reacts too strongly to the idiosyncrasies of the data). This high variance leads to poor generalization.





Strategies to Prevent Overfitting:

- **Regularization (L1/L2)**
- **Dropout**
- **Early Stopping**
- **Cross-Validation**
- **Simplify the Model**
- **Increase Data**
- **Noise Reduction**





Layers of Feed Forward Neural Networks

Feed Forward Neural Network Layers





Backpropagation

What is Backpropagation?

Backpropagation is a powerful algorithm in deep learning, primarily used to train artificial neural networks, particularly feed-forward networks. It works iteratively, minimizing the cost function by adjusting weights and biases

In simpler terms, backpropagation uses the chain rule to calculate the rate at which loss changes in response to any change to a specific weight (or bias) in the network. Generally speaking, training neural networks with backpropagation entails the following steps: A forward pass, making predictions on training data.





Neural networks consist of neurons that operate using **weights**, **biases**, and **activation functions**. In the learning process, these weights and biases are updated based on the error produced at the output—a process known as **backpropagation**. Activation functions enable backpropagation by providing gradients that are essential for updating the weights and biases.

Activation Function

An activation function is a mathematical function applied to the output of a neuron. It introduces non-linearity into the model, allowing the network to learn and represent complex patterns in the data. Without this non-linearity feature, a neural network would behave like a linear regression model, no matter how many layers it has.

The activation function decides whether a neuron should be activated by calculating the weighted sum of inputs and adding a bias term. This helps the model make complex decisions and predictions by introducing non-linearities to the output of each neuron.



An **activation function** in a neural network defines the output of a neuron based on its input. It introduces non-linearity into the network, enabling it to learn complex patterns. Common activation functions include:

Sigmoid: Maps the input to a range between 0 and 1, commonly used for binary classification.

Tanh: Maps the input to a range between -1 and 1, often used in hidden layers.

ReLU (Rectified Linear Unit): Outputs the input directly if it's positive, and 0 if it's negative. It's commonly used due to its simplicity and efficiency in training deep networks.





1. Sigmoid Activation Function

The **sigmoid function** is one of the earliest activation functions used in neural networks.

Advantages:

Sigmoid is differentiable and smooth, making it suitable for backpropagation. Its output is in the range $(0, 1)$, which is especially useful for binary classification problems. It works well in shallow networks with few layers.

Limitations:

Vanishing Gradient Problem: For large positive or negative inputs, the gradient of the sigmoid function becomes very small, slowing down the learning process.

Non-zero centered output: Since the output is between 0 and 1, this can cause issues during optimization by leading to slower convergence.





2. ReLU Activation Function

The ReLU (Rectified Linear Unit) is the most widely used activation function in deep neural networks today.

It is defined as: $\text{ReLU}(x) = \max(0, x)$

Advantages:

Computationally efficient: It is simple to compute, with only a comparison operation, making it faster than sigmoid and tanh.

No Vanishing Gradient: Unlike sigmoid and tanh, ReLU doesn't suffer from the vanishing gradient problem. For positive inputs, the gradient is constant, which speeds up learning.

Sparsity: ReLU sets negative values to zero, which leads to sparsity in the neural network, helping with model generalization.

Limitations:

Dying ReLU Problem: In some cases, neurons can become inactive and always output zero for any input, effectively "dying" and not learning anymore. This happens when a large number of inputs result in negative values.

Unbounded output: ReLU outputs can become very large, which may cause exploding gradients.



3. Tanh Activation Function

The **tanh function** is mathematically similar to the sigmoid function but differs in its output range.

Advantages:

The output is zero-centered, meaning it maps negative inputs strongly negative and positive inputs strongly positive. This property allows for faster convergence during optimization.

Unlike sigmoid, the output of tanh is between -1 and 1, which helps avoid issues caused by extreme values.

Limitations:

Like sigmoid, tanh suffers from the vanishing gradient problem when the input is large in magnitude.





MIT School of Computing

Department of Computer Science & Engineering



MIT-ADT
UNIVERSITY
PUNE, INDIA

NOTE –
For numericals refer Classnotes.
Mapreduce topic –refer from own notes of participative learning (practice good examples in details).

